

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LFCX:Datenaustauschformate

[→ ZP:Sheet:2]:

Die Frage nach Daten-(Datei)-Austauschformaten taucht in der Ausbildung zur Fachinformatikerin an verschiedenen Stelle auf. Es geht dabei um Erfassung strukturierter Daten in Dateien - insbesondere im CSV-, INI-, JSON-, XML- und YAML-Format.

Die erläutern wir anhand derselben Referenzdaten:

1.) Referenzdaten ([→ ZP:Sheet:3])

Messdatum:	2025-03-01	2025-03-02	2025-03-03	2025-03-04
Systole	182	158	179	170
Puls	66	75	67	78

2.) CSV ([→ ZP:Sheet:4])

= Comma-separated values

Charakteristika:

- Zeilen bilden einen Datensatz = Messung.
- Ein Datensatz besteht aus Datenfeldern.
- Datenwerte können - abgesehen von den Sonderzeichen - beliebige Zeichen enthalten.
- Der erste Datensatz **kann** ein Kopfdatensatz sein, der die Spaltennamen definiert
- Gut geeignet zur Erfassung von sich wiederholenden Messdatensätzen

Sonderzeichen:

1. **Datensatztrenner** = meistens **LF** (LNX) bzw. **CR LF** (WIN)
2. **Datenfeldtrenner** = oft **,** (andere sind möglich, etwa **;**, **:**, **TAB**, **...**)

3. **Datenfeldbegrenzer** = Anführungszeichen vorne und hinten ‘

Beispiel:

```
1 Datum,Systole,Puls
2 2025-03-01, 182, 66
3 2025-03-02, 158, 75
4 2025-03-03, 179, 67
5 2025-03-04, 170, 78
```

vgl.

- [https://de.wikipedia.org/wiki/CSV_\(Dateiformat\)](https://de.wikipedia.org/wiki/CSV_(Dateiformat))
 - https://en.wikipedia.org/wiki/Comma-separated_values
-

3.) INI ([→ ZP:Sheet:5])

= Initialisierungsdatei

- von Microsoft für die Windowskonfiguration eingeführt
- von vielen anderen ‘Systemen’ zu vielen anderen Zwecken übernommen - gelegentlich mit veränderter Syntax

Charakteristika:

- Jede Zeile ist
 - ein Schlüsselwertpaar oder
 - eine Sektionsüberschrift
- Ein Schlüssel ist ein String ohne innere Leerzeichen (oder andere Whitespacezeichen)
- Ein Wert ist String ohne Blanks oder ein mit Anführungszeichen begrenzter String mit Blanks
- Gut geeignet zur Erfassung / Strukturierung von einem Datensatz zur Konfiguration
- Nur mit Verrenkungen geeignet für sich wiederholende Messungen

Sonderzeichen:

1. **Konfigurationstrenner** = meistens LF (LNX) bzw. CR LF (WIN) (könnte auch ein Blank sein)
2. [und] zur Markierung einer Sektionsüberschrift
3. **Werbegrenzer** = Anführungszeichen vorne und hinten
4. ; als Kommentareinleiter

Semantische Regeln:

- Sektionsbezeichner müssen unique sein.
- In einer Sektion müssen die Schlüssel unique sein.
- Kommentare dürfen nur auf separaten Zeilen
- Groß- und Kleinschreibung nicht unterschieden

Beispiel:

```
1 [ Blutdruck ]
2 Systole=mmHg
3 Diastole=mmHg
4 ; 'mmHg' = Millimeter Quecksilbersäule
5 [ Puls]
6 Frequenz=bpm
7 ; 'bpm' = Nits per Minute
```

versus

```
1 [2025-03-01]
2 Systole=182
3 Puls=66
4 [2025-03-02]
5 Systole=158
6 Puls=75
7 [2025-03-03]
8 Systole=179
9 Puls=67
10 [2025-03-04]
11 Systole=170
12 Puls=78
```

Hinweis: Sektionen dürfen üblicherweise NICHT verschachtelt sein.

vgl. <https://de.wikipedia.org/wiki/Initialisierungsdatei>

4.) JSON ([→ ZP:Sheet:6])

= JavaScript Object Notation

Charakteristika:

- kennt die Elemente
 - **null** (Nullwerte)
 - **true** bzw. **false** (Boolsche Werte)
 - **[+, -] [0-9] [0-9]* (. [0-9] [0-9]*)**
 - (Unicode) *Zeichenkette* mit oder ohne Escapesequenzen und eingefasst mit **"**
 - *Array* beginnend mit **[** und endend mit **]**.

- *Object* beginnend mit { und endend mit }.
- besteht aus einem Schlüsselwertpaar mit innerem Doppelpunkt: **Schlüssel** : **Wert**
 - *Schlüssel* ist eine Zeichenkette
 - *Wert* ist ein beliebiges Element
- Aufeinander folgende Schlüsselwertpaare werden durch Kommata abgetrennt.
- Whitespaces haben keine Bedeutung und können beliebig oft eingeschoben werden.

Sonderzeichen: :, ", \, [und], { und }

Semantische Regeln:

- Schlüssel müssen unique sein

Beispiel:

```
1  [  
2    { "2025-03-01":{  
3      "Systole": 182,  
4      "Puls": 66  
5    } } ,  
6    { "2025-03-02":{  
7      "Systole": 158,  
8      "Puls": 75  
9    } } ,  
10   { "2025-03-03":{  
11     "Systole": 179,  
12     "Puls": 67  
13   } } ,  
14   { "2025-03-04":{  
15     "Systole": 170,  
16     "Puls": 78  
17   } }  
18 ]
```

- vgl. <https://de.wikipedia.org/wiki/JSON>

Disclaimer: ([→ ZP:Sheet:7])

- JSON ist in der 2. Version aus dem Dezember 2017 als **JSON data interchange syntax** von der ECMA (= *international industry association for standardization of information and communication systems*) unter der Nummer 404 als **JSON data interchange syntax** standardisiert. Das zugehörige Standardisierungsdokument ist als PDF frei herunterladbar.
 - <https://ecma-international.org/publications-and-standards/standards/ecma-404/>
 - https://ecma-international.org/wp-content/uploads/ECMA-404_2nd_edition_december_2017.pdf

- <https://ecma-international.org/>
- Die JSON-Homepage bietet selbst eine darauf aufbauende Syntaxspezifikation in Form einer visualisierten Backus-Naur-Form
 - <https://www.json.org/json-de.html>
 - <https://de.wikipedia.org/wiki/Backus-Naur-Form>
 - https://de.wikipedia.org/wiki/Erweiterte_Backus-Naur-Form
- Hier eine kondensierte Version als EBNF-Spezifikation:

```
1 (R1) json = element ;
2 (R2) element = ws value ws ;
3 (R3) value = object | array | string | number | 'true' | 'false' | '
    null';
4 (R4) object = '{' ws '}' | '{' ws members ws '}';
5 (R5) members = member | member ',' members;
6 (R6) member = ws string ws ':' element;
7 (R7) array = '[' ws ']' | '[' elements ']';
8 (R8) elements = element | element ',' elements;
9 (R9) ws = ' ' | '\0020' ws | '\000A' ws | '\000D' ws | '\0009' ws;
```

Aus R1 - R3 folgt, dass jede Datei, die mit einem korrekt notierten Object, Array etc. beginnt, eine korrekte JSON-Datei ist.

Gelegentlich hört man trotzdem, eine JSON-Datei müsse mit einer geschweiften Klammer beginnen. Das ist falsch.

ÜBUNG CRX:Datenaustauschformate:01**

- ☐ Welche der folgenden JSON-Dateien ist nicht 'wohlgeformt' und warum?

1. { }
2. []
3. "stimme": "hoch"
4. true

Lösung: ([→ ZP:Sheet:R8])

- [1] folgt aus R1 + R2 + R3 + R4 + R8
- [2] folgt aus R1 + R2 + R3 + R7 + R8

- [3] keine Ableitung
 - [4] folgt aus R1 + R2 + R3
-

ÜBUNG CRX:Datenaustauschformate:02**

- ☐ Die 'kondensierte' EBNF auf [→ ZP:Sheet:7] ist unvollständig. Für welches zentrale Syntagma habe ich die Ersetzungsregel "vergessen"?
 - ☐ Sehen Sie unter <https://www.json.org/json-de.html> nach, wie die Regel heißen müsste.
-

Lösung:

- Für das Syntagma *elements* in `array = '['ws ']' | '['elements ']'`; gibt es keine Ersetzungsregel.
- Dort findet man die Regel `elements = element | element ','elements`

5.) XML ([→ ZP:Sheet:R9])

= Extensible Markup Language

Charakteristika:

- Jeder Wert wird durch ein öffnendes und ein schließendes Tag ausgezeichnet
 - Jedes Tag beginnt mit einem < und endet mit einem >
 - Beim schließenden Tag folgt auf das öffnende < ein Backslash /
 - Der eigentliche Text innerhalb eines Tags beginnt mit einem Zeichen, gefolgt von beliebig vielen Zeichen und Zahlen.
 - Der eigentliche Text innerhalb eines Tags enthält keine WhiteSpace-Zeichen.
 - Der eigentliche Text innerhalb des öffnenden und schließenden Tags muss gleich sein.
 - Jede XML-Datei beginnt mit `<?xml version="1.1"encoding="UTF8">`.
 - Danach folgen beliebig viele getaggte und verschachtelte Werte
 - Vor und nach einem Tag können beliebig viele Whitespaces folgen.
 - Whitespaces zwischen den Tags gehören zum Wert. Es ist gute Tradition, Strings trotzdem mit Hochkommata zusammenzufassen.

Sonderzeichen: * & * < * > * "

Beispiel:

```
1 <?xml version="1.1" encoding="UTF9">
2 <DemoMessReihe>
3   <D20250301>
4     <Systole>182</Systole>
5     <Puls>66</Puls>
6   </D20250301>
7   <D20250302>
8     <Systole>158</Systole>
9     <Puls>75</Puls>
10  </D20250302>
11  <D20250303>
12    <Systole>179</Systole>
13    <Puls>67</Puls>
14  </D20250303>
15  <D20250304>
16    <Systole>170</Systole>
17    <Puls>78</Puls>
18  </D20250304>
19 </DemoMessReihe>
```

Anmerkung: Welche Tags und welche Typen der Werte in einem XML-Dateityp erlaubt sind, werden durch externe 'Doc-Type-Definition'-Dateien (DTD) oder xml-Schema-Dateien bestimmt.

Zu den vielen anderen Detailanforderungen/Möglichkeiten vgl.

- https://de.wikipedia.org/wiki/Extensible_Markup_Language
- Robert Eckstein: XML Pocket Reference, O'Reily, 2001
- u. viele andere neuere Werke

6. YAML ([→ ZP:Sheet:10])

steht für *YAML Ain't Markup Language* (ursprünglich: *Yet Another Markup Language*))

Charakteristika:

- besteht aus Blöcken, jeweils abgetrennt durch ---
- Hat die Datei nur einen Block, werden die Trenner weggelassen
- Ein Block kann 1-n Schlüssel-Wert-Paare enthalten:
 - Schlüssel und Wert sind durch ':' getrennt
 - Ein Block kann in einer Zeile geschrieben werden:
 - Diese Zeile beginnt mit { und endet mit }
 - Die Schlüsselwert-Paare werden dann durch Kommata getrennt.
- Ein Block kann eine assoziative Liste enthalten

Sonderzeichen:

vgl. <https://de.wikipedia.org/wiki/YAML>

Beispiel:

```
1
2 - D20250301: {Systole: 182, Puls: 66}
3 - D20250302:
4   Systole: 158
5   Puls: 75
6 ---
7 Messreihe-2:
8 - D20250303: {Systole: 179, Puls: 67}
9 - D20250304:
10   Systole: 170
11   Puls: 78
12 ---
```

Hinweis:

- YAML-Dateien werden oft als Konfigurationsdateien im Cloud-Kontext verwendet.
- Waren gedacht als Vereinfachung der JSON-Syntax
- Ist aber durch die Variantenvielfalt schwer einheitlich / projektübergreifend zu nutzen.

vgl. <https://de.wikipedia.org/wiki/YAML#Kritik>